

OPEN IT SUMMER

BOOTCAMP

DAY 6

Databases & PostgreSQL

Relational Model • Schema Design • SQL • JOINS • Indexes • Normalization

Duration: ~2 hrs | PostgreSQL setup → Schema → Queries → pgAdmin testing

Bootcamp Roadmap — Where Day 6 Fits

Day 2: C# + API (static list) → Day 6: DB concepts + SQL → Day 7: EF Core connects your API to PostgreSQL

DATABASES & POSTGRESQL

Session Overview

Day 6 of 15

01

Why a Database? — From _books List to PostgreSQL

The problem with in-memory data

02

The Relational Model

Tables, rows, columns, keys, relationships

03

PostgreSQL Setup & pgAdmin

Install, connect, and verify

04

Data Types & Schema Design

int, varchar, boolean, timestamp, serial

05

SQL — CREATE, INSERT, SELECT

Writing your first queries

06

WHERE, ORDER BY, LIMIT & Filtering

Reading specific data

07

JOINS — Connecting Tables

INNER, LEFT, combining related data

08

Aggregates, GROUP BY & Indexes

COUNT, SUM, AVG + performance

09

2-Hour Hands-On Lab

Design + build + query your own schema

Why Do We Need a Database?

Fixes Day 2 limitation

The problem with the static `_books` list from Day 2

Data Dies on Restart

Every time you stop dotnet run, your `_books` list resets to the 5 seeded books. POST a new book, restart — it is gone.

No Multi-User Support

Two people using the API at the same time get different lists if the static field ever diverges. Shared mutable state causes bugs.

No Real Querying

You wrote LINQ filters by hand. A real DB can search millions of rows in milliseconds using indexes. LINQ on a list is $O(n)$ — slow.

No Relationships

A Book has an Author. With a list you store the author as a string. A database stores Authors in their own table and links them properly.

No Persistence

The list lives in RAM. RAM is lost on crash, restart, or deployment. A database writes to disk — data survives anything short of hardware failure.

No Scale

A static list in one server cannot be shared with a second server. Databases are external services — multiple API instances connect to the same DB.

Day 7: your `BooksController` stops using `_books` and starts using EF Core + PostgreSQL. Same endpoints, same Postman requests — real persistence.

The Relational Model — Core Concepts

Edgar Codd, 1970. Every modern database is built on these ideas.

A Table = rows + columns with rules

```
-- A table is like a C# class definition:  
--   columns = properties  
--   rows    = individual objects  
--   rules   = data type + constraints
```

```
-- This table stores books
```

```
SELECT * FROM books;
```

id	title	author	available	year
1	Clean Code	Robert Martin	true	2008
2	C# in Depth	Jon Skeet	false	2019
3	Dune	Frank Herbert	true	1965

```
-- Compare to your Day 2 C# class:
```

```
-- public class Book {  
--     public int    Id        { get; set; }  
--     public string Title     { get; set; }  
--     public string Author    { get; set; }  
--     public bool   Available { get; set; }  
-- }
```

```
-- The mental model is IDENTICAL.
```

```
-- Table  = class definition
```

```
-- Row    = one object instance
```

```
-- Column = one property
```

Key Concepts

Table (Relation)

A named grid of rows and columns. One table per entity type (books, authors, loans).

Row (Record / Tuple)

One single entry in the table. Equivalent to one C# object instance.

Column (Attribute)

A named, typed field. Every row has the same columns. Equivalent to a C# property.

Primary Key (PK)

A column (or combination) that uniquely identifies each row. Usually "id SERIAL". No two rows can share a PK.

Foreign Key (FK)

A column that references the PK of another table. Creates a relationship. `author_id` in `books` references `id` in `authors`.

Schema

The overall structure — all tables, their columns, types, and relationships. Like a C# class definition but for the database.

PostgreSQL Setup — Install & Connect

 Required for Day 7

Everything you need running before you can write SQL

Install PostgreSQL

```
# Windows – download the installer:
# postgresql.org/download/windows
# → Download → PostgreSQL 16 → Run installer
# → Remember the password you set for "postgres"
# → Default port: 5432 (leave as-is)
# → Default locale: leave as-is → Finish
```

Connect with psql (Terminal)

```
# Open psql command-line client:
# Windows: search "psql" in Start Menu
# Mac/Linux: open Terminal

psql -U postgres
# Enter the password you set during install

# You should see:
# postgres=#

# Create a database
CREATE DATABASE librarydb;
# Connect to it:
\c librarydb
# You should see:
# librarydb=#
# List all databases:
\l
# Quit psql:
\q
```

pgAdmin — Visual GUI (Easier for Beginners)

```
# pgAdmin is installed automatically with PostgreSQL on Windows.
# Mac: download from pgadmin.org/download/

# To connect in pgAdmin:
# 1. Open pgAdmin → Servers → right-click → Register → Server
# 2. Name: "Local"
# 3. Connection tab: Host = localhost | Port = 5432 | Username = postgres | Password = (your password)
# 4. Click Save → expand Servers → Local → Databases → librarydb → Schemas → Tables
```

Port 5432 is PostgreSQL's default. Remember your postgres password — you need it every time you connect and later in appsettings.json on Day 7.

PostgreSQL Data Types

Choosing the right type — it affects storage, performance, and validation

Numeric & Boolean

```
-- Integers
SMALLINT      -- -32,768 to 32,767 (rarely used)
INTEGER / INT  -- ±2.1 billion      (most common)
BIGINT        -- ±9.2 quintillion (large counts)

-- Auto-increment integers (use for PRIMARY KEY)
SERIAL         -- auto-increment INT      (1, 2, 3...)
BIGSERIAL      -- auto-increment BIGINT

-- Decimals
NUMERIC(p, s)  -- exact: p=total digits, s=decimal places
DECIMAL(10, 2) -- e.g. 99999999.99 — USE FOR MONEY
REAL           -- 6 decimal digits, approximate
DOUBLE PRECISION -- 15 decimal digits, approximate

-- Boolean
BOOLEAN        -- TRUE or FALSE (also: 't','f','yes','no','1','0')
```



```
-- C# equivalents:
-- INT      → int
-- BIGINT   → long
-- NUMERIC  → decimal (use for money!)
-- BOOLEAN  → bool
```

Text, Date & Other

```
-- Text
CHAR(n)        -- fixed length, padded with spaces
VARCHAR(n)     -- variable up to n characters
TEXT           -- unlimited length (no performance penalty in PG)

-- Dates and Times
DATE           -- date only: 2026-05-10
TIME           -- time only: 14:30:00
TIMESTAMP      -- date + time: 2026-05-10 14:30:00
TIMESTAMPTZ    -- timestamp WITH timezone (recommended)

-- Special
UUID           -- globally unique ID: a1b2c3d4-...
JSON           -- stores JSON text
JSONB          -- stores JSON binary (faster querying)
ARRAY          -- e.g. INTEGER[], TEXT[]

-- C# equivalents:
-- VARCHAR   → string
-- TEXT      → string
-- BOOLEAN   → bool
-- TIMESTAMP → DateTime
-- UUID      → Guid

-- RULE: Use TEXT over VARCHAR unless you have a
-- specific reason to enforce a length limit.
-- Use TIMESTAMPTZ over TIMESTAMP — always store UTC.
```

SERIAL is shorthand for: INTEGER NOT NULL DEFAULT nextval('sequence'). It auto-increments. Always use it for primary keys in Day 2–style projects.

CREATE TABLE & Constraints

Defining the structure and rules of your data — equivalent to your C# model classes

Library Schema — CREATE TABLE statements

```
-- Authors table (create FIRST — books depends on it)
CREATE TABLE authors (
  id          SERIAL          PRIMARY KEY,          -- auto-increment, unique, never null
  full_name   VARCHAR(150)    NOT NULL,            -- required, max 150 chars
  nationality  VARCHAR(100),    -- optional
  birth_year  INT              CHECK (birth_year > 1800 AND birth_year < 2010)
);
-- ↑ CHECK enforces a range, like [Range] in C#

-- Books table (references authors)
CREATE TABLE books (
  id          SERIAL          PRIMARY KEY,
  title       VARCHAR(255)    NOT NULL,
  genre       VARCHAR(80),
  year        INT              CHECK (year >= 1000 AND year <= 2030),
  available   BOOLEAN          NOT NULL DEFAULT TRUE, -- default value when not specified
  author_id   INT              REFERENCES authors(id) ON DELETE SET NULL
);
-- ↑ FOREIGN KEY — links to authors.id
-- ON DELETE SET NULL: if the author is deleted, set author_id to NULL
-- (other options: CASCADE = delete the book too, RESTRICT = block deletion)

-- Loans table — tracks who borrowed which book
CREATE TABLE loans (
  id          SERIAL          PRIMARY KEY,
  book_id     INT              NOT NULL REFERENCES books(id) ON DELETE CASCADE,
  member_name VARCHAR(150)    NOT NULL,
  loaned_at   TIMESTAMPTZ     NOT NULL DEFAULT NOW(), -- auto-set to current timestamp
  returned_at TIMESTAMPTZ     -- NULL means not yet returned
);
```

Constraints Quick Reference

PRIMARY KEY	Unique + NOT NULL. Only one per table. The row's identity.	NOT NULL	Value must be provided. Like [Required] in C#.
UNIQUE	No two rows can have the same value in this column.	CHECK (expr)	Value must satisfy a condition. Like [Range] in C#.
DEFAULT value	Value used when INSERT does not specify this column.	REFERENCES	Foreign key — value must exist in another table's PK.

INSERT & SELECT — Writing and Reading Data

The two most common SQL operations you will write every day

INSERT — Adding rows

```
-- INSERT single row
INSERT INTO authors (full_name, nationality, birth_year)
VALUES ('Robert Martin', 'American', 1952);

-- INSERT multiple rows at once
INSERT INTO authors (full_name, nationality)
VALUES
    ('Jon Skeet', 'British'),
    ('Frank Herbert', 'American'),
    ('Yuval Harari', 'Israeli'),
    ('David Thomas', 'British');

-- INSERT into books (referencing author ids)
INSERT INTO books (title, genre, year, available, author_id)
VALUES
    ('Clean Code', 'Programming', 2008, TRUE, 1),
    ('C# in Depth', 'Programming', 2019, FALSE, 2),
    ('Dune', 'Fiction', 1965, TRUE, 3),
    ('Sapiens', 'History', 2011, TRUE, 4),
    ('The Pragmatic Programmer', 'Programming', 1999, FALSE, 5);

-- INSERT and get the new row back immediately
INSERT INTO books (title, genre, year, author_id)
VALUES ('Clean Architecture', 'Programming', 2017, 1)
RETURNING id, title; -- ← returns the new row's id and title
```

SELECT — Reading rows

```
-- All rows, all columns
SELECT * FROM books;

-- Specific columns only (always prefer this over *)
SELECT id, title, available FROM books;

-- With an alias – rename columns in the result
SELECT
    id,
    title AS book_title,
    available AS is_available
FROM books;

-- Limit results – first 3 books only
SELECT * FROM books LIMIT 3;

-- Offset – skip first 2, then take 3 (pagination)
SELECT * FROM books
LIMIT 3 OFFSET 2;
-- Offset 0 = page 1, Offset 3 = page 2, etc.

-- Distinct – no duplicate values
SELECT DISTINCT genre FROM books;
-- → Programming, Fiction, History

-- Count total rows
SELECT COUNT(*) FROM books;

-- RETURNING after INSERT (get the new id back)
INSERT INTO books (title, author_id)
VALUES ('New Book', 1)
```

Always list specific columns in SELECT (not *) in production code — it prevents issues when the table schema changes and makes queries self-documenting.

WHERE, ORDER BY, UPDATE & DELETE

Filtering, sorting, and modifying data

WHERE — Filtering rows

```
-- Exact match
SELECT * FROM books WHERE available = TRUE;

-- Comparison operators: = <> < > <= >=
SELECT * FROM books WHERE year > 2000;
SELECT * FROM books WHERE year BETWEEN 1990 AND 2010;

-- Text matching
SELECT * FROM books WHERE genre = 'Programming';

-- Case-insensitive search (ILIKE = case-insensitive LIKE)
SELECT * FROM books WHERE title ILIKE '%clean%';
-- % = wildcard: matches any characters
-- 'clean code' ILIKE '%clean%' → TRUE
-- 'CLEAN CODE' ILIKE '%clean%' → TRUE (case-insensitive)

-- Multiple conditions
SELECT * FROM books
WHERE available = TRUE AND genre = 'Programming';

SELECT * FROM books
WHERE genre = 'Fiction' OR genre = 'History';

-- IN — match any value in a list
SELECT * FROM books
WHERE genre IN ('Fiction', 'History', 'Science');

-- NULL checks (never use = NULL — always use IS NULL)
SELECT * FROM loans WHERE returned_at IS NULL;    -- not returned yet
SELECT * FROM loans WHERE returned_at IS NOT NULL; -- already returned

-- ORDER BY
SELECT * FROM books ORDER BY title ASC;           -- alphabetical
SELECT * FROM books ORDER BY year DESC;           -- newest first
SELECT * FROM books ORDER BY genre ASC, year DESC; -- multiple columns
```

UPDATE & DELETE

```
-- UPDATE — change existing rows
UPDATE books
SET available = FALSE
WHERE id = 3;

-- Update multiple columns at once
UPDATE books
SET
    title     = 'Clean Code (2nd Ed)',
    year      = 2024,
    available = TRUE
WHERE id = 1;

-- ALWAYS include WHERE on UPDATE
-- Without WHERE: updates EVERY row in the table!
-- UPDATE books SET available = FALSE; ← ALL books unavailable!

-- DELETE — remove rows
DELETE FROM books WHERE id = 5;

-- ALWAYS include WHERE on DELETE
-- Without WHERE: deletes EVERY row!
-- DELETE FROM books; ← EMPTY TABLE!

-- Soft delete pattern (preferred in production)
-- Instead of actually deleting, mark as deleted:
-- ALTER TABLE books ADD COLUMN deleted_at TIMESTAMPTZ;
-- UPDATE books SET deleted_at = NOW() WHERE id = 5;
-- SELECT * FROM books WHERE deleted_at IS NULL;

-- RETURNING works on UPDATE and DELETE too
UPDATE books
SET available = FALSE
WHERE id = 3
RETURNING id, title, available;
```

ALWAYS use WHERE with UPDATE and DELETE. Running them without WHERE in production is one of the most common and costly database mistakes.

Create & Populate the Library Schema

SETUP: Open pgAdmin → connect to PostgreSQL → open Query Tool on librarydb

PART A — Create tables (run in this order):

1. CREATE TABLE authors with: id (SERIAL PK), full_name (VARCHAR 150, NOT NULL), nationality (VARCHAR 100), birth_year (INT with CHECK > 1800)
2. CREATE TABLE books with: id (SERIAL PK), title (VARCHAR 255, NOT NULL), genre (VARCHAR 80), year (INT with CHECK 1000–2030), available (BOOLEAN NOT NULL DEFAULT TRUE), author_id (INT REFERENCES authors(id) ON DELETE SET NULL)
3. CREATE TABLE loans with: id (SERIAL PK), book_id (INT NOT NULL REFERENCES books(id)), member_name (VARCHAR 150 NOT NULL), loaned_at (TIMESTAMPTZ DEFAULT NOW()), returned_at (TIMESTAMPTZ) ← nullable means not yet returned

PART B — Seed data:

4. INSERT 4 authors: Robert Martin (American), Jon Skeet (British), Frank Herbert (American), [Your Name](Filipino)
5. INSERT 5 books linking to the correct author_id

PART C — Verify with SELECT:

6. SELECT * FROM authors; → should show 4 rows
7. SELECT * FROM books; → should show 5 rows
8. SELECT COUNT(*) FROM books; → should show 5

JOINS — Combining Data from Multiple Tables

The most powerful feature of relational databases

A JOIN lets you query data from two or more tables at once by linking them on a shared column — usually a foreign key.

INNER JOIN — only matching rows

```
-- INNER JOIN: returns rows where BOTH tables have a match
-- Books without an author_id (NULL) are EXCLUDED

SELECT
  b.id,
  b.title,
  b.genre,
  b.year,
  a.full_name AS author,
  a.nationality AS author_country
FROM books AS b
INNER JOIN authors AS a
  ON b.author_id = a.id
WHERE b.available = TRUE
ORDER BY b.year DESC;

-- "b" is an alias for books
-- "a" is an alias for authors
-- ← the JOIN condition

-- Result:
-- id | title      | genre      | year | author      | author_country
-- 4  | Sapiens    | History    | 2011 | Yuval Harari | Israeli
-- 1  | Clean Code | Programming| 2008 | Robert Martin | American
-- 3  | Dune       | Fiction    | 1965 | Frank Herbert | American

-- The key: b.author_id = a.id
-- "match books.author_id with authors.id"
-- This is exactly the foreign key relationship you defined in CREATE TABLE
```

INNER JOIN = intersection (only matching rows on both sides)

LEFT JOIN & Multiple JOINS

```
-- LEFT JOIN: ALL rows from the left table,
-- matched rows from right (NULL if no match)
-- Use when: you want books even if they have no author

SELECT
  b.title,
  a.full_name AS author
FROM books AS b
LEFT JOIN authors AS a ON b.author_id = a.id;

-- A book with no author gets: author = NULL
-- INNER JOIN would exclude that book entirely

-- JOIN three tables at once
SELECT
  b.title,
  a.full_name AS author,
  l.member_name,
  l.loaned_at
FROM loans AS l
INNER JOIN books AS b ON l.book_id = b.id
INNER JOIN authors AS a ON b.author_id = a.id
WHERE l.returned_at IS NULL -- currently borrowed
ORDER BY l.loaned_at DESC;

-- Result: all active loans with book title and author name
-- Notice: you can chain as many JOINS as needed
```

LEFT JOIN = all from left + matching from right (NULLs on right if no match)

Aggregate Functions & GROUP BY

Summarising and grouping data — like LINQ's .Count(), .Average(), .GroupBy()

Aggregate Functions

```
-- COUNT — how many rows
SELECT COUNT(*)      FROM books;           -- all rows: 5
SELECT COUNT(*)      FROM books WHERE available = TRUE; -- 3
SELECT COUNT(author_id) FROM books;        -- non-NULL author_ids only

-- SUM, AVG, MIN, MAX
SELECT
    COUNT(*)          AS total_books,
    SUM(year)         AS sum_of_years, -- not useful here, but valid
    ROUND(AVG(year), 0) AS average_year,
    MIN(year)         AS oldest_year,
    MAX(year)         AS newest_year
FROM books;

-- Result:
-- total_books | average_year | oldest_year | newest_year
-- 5           | 2000         | 1965        | 2019

-- Aggregate on joined data
SELECT
    a.full_name,
    COUNT(b.id)      AS books_written,
    MIN(b.year)      AS first_book,
    MAX(b.year)      AS latest_book
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.id
GROUP BY a.id, a.full_name
ORDER BY books_written DESC;
```

GROUP BY & HAVING

```
-- GROUP BY — group rows with the same value
-- then apply aggregate per group

-- Count books per genre
SELECT
    genre,
    COUNT(*) AS total
FROM books
GROUP BY genre
ORDER BY total DESC;

-- Result:
-- genre      | total
-- Programming | 3
-- Fiction    | 1
-- History    | 1

-- HAVING — filter GROUPS (like WHERE but for aggregates)
-- WHERE filters rows BEFORE grouping
-- HAVING filters groups AFTER grouping

SELECT
    genre,
    COUNT(*) AS total
FROM books
GROUP BY genre
HAVING COUNT(*) > 1; -- only genres with more than 1 book
-- Result: just Programming (3 books)

-- Combined: WHERE + GROUP BY + HAVING
SELECT
    genre,
    COUNT(*) AS total,
    AVG(year) AS avg_year
FROM books
WHERE available = TRUE -- filter rows first
GROUP BY genre        -- then group
HAVING COUNT(*) >= 1  -- then filter groups
ORDER BY total DESC;
```

Rule: WHERE filters individual rows. HAVING filters the result of GROUP BY. You cannot use aggregate functions (COUNT, SUM) inside WHERE — use HAVING instead.

Constraints & Indexes

Data integrity rules and query performance

Constraints in Detail

```
-- PRIMARY KEY (unique + not null, auto-indexed)
id SERIAL PRIMARY KEY,

-- NOT NULL
title VARCHAR(255) NOT NULL,

-- UNIQUE – no duplicates in this column
email VARCHAR(255) UNIQUE,

-- CHECK – custom validation rule (like [Range] in C#)
year INT CHECK (year >= 1000 AND year <= 2030),
gpa NUMERIC CHECK (gpa >= 0.0 AND gpa <= 4.0),

-- FOREIGN KEY – two ways to write it:
-- Inline:
author_id INT REFERENCES authors(id),

-- Or as a named table constraint (better for complex FKs):
CONSTRAINT fk_books_author
    FOREIGN KEY (author_id)
    REFERENCES authors(id)
    ON DELETE SET NULL,

-- Add a UNIQUE constraint to an existing table:
ALTER TABLE books
ADD CONSTRAINT uq_books_isbn UNIQUE (isbn);

-- Add a CHECK to an existing table:
ALTER TABLE books
ADD CONSTRAINT chk_year CHECK (year >= 1000);
```

Indexes — Query Performance

```
-- Without an index: PostgreSQL scans every row (slow)
-- With an index: PostgreSQL jumps directly (fast)

-- B-tree index (default) – for =, <, >, BETWEEN, ORDER BY
CREATE INDEX idx_books_genre
    ON books (genre);

CREATE INDEX idx_books_author_id
    ON books (author_id); -- always index foreign keys!

-- Unique index – enforces uniqueness AND improves speed
CREATE UNIQUE INDEX idx_books_isbn
    ON books (isbn);

-- Partial index – only index a subset of rows
CREATE INDEX idx_available_books
    ON books (title)
    WHERE available = TRUE;
-- Only indexes available books – smaller, faster

-- See all indexes on a table (in psql):
\d books

-- When to add an index:
-- ✓ Columns used in WHERE frequently
-- ✓ Foreign key columns (always!)
-- ✓ Columns used in ORDER BY
-- ✗ NOT on columns that change very frequently
-- ✗ NOT on small tables (full scan may be faster)
```

ON DELETE Behavior — What Happens When a Referenced Row is Deleted

```
ON DELETE RESTRICT    -- block the delete if any books reference this author (safest – prevents accidental data loss)
ON DELETE CASCADE     -- automatically delete the books too when the author is deleted (use carefully!)
ON DELETE SET NULL    -- set author_id = NULL on the book (book stays, loses its author link)
ON DELETE SET DEFAULT -- set author_id to the column default value
```

Normalization — Designing Good Schemas

Rules for eliminating redundancy and preventing update anomalies

❌ BAD — Unnormalized (all in one table)

```
-- All data in ONE table — looks simple but causes problems:
CREATE TABLE books_bad (
  id          SERIAL PRIMARY KEY,
  title       TEXT,
  author_name TEXT,      -- ← duplicated for every book by this author
  author      author,
  author_country TEXT,   -- ← duplicated for every book by this author
  author_birth INT,      -- ← duplicated for every book by this author
  genre       TEXT,
  year        INT
);

-- Problems with this design:
-- 1. If Robert Martin changes nationality, you must UPDATE
--    every single one of his books. Miss one → inconsistent data.
-- 2. You cannot store an author without a book.
-- 3. If you delete his last book, you lose his info entirely.
-- 4. Storage waste — author_name repeated 10 times for 10 books.

-- These are called "update anomalies" and "insertion anomalies".
-- Normalization prevents them by putting data
-- in the right table.
```

✅ GOOD — Normalized (3 Normal Forms)

```
-- Separate tables — each fact stored once:

CREATE TABLE authors (      -- author data lives HERE only
  id          SERIAL PRIMARY KEY,
  full_name   VARCHAR(150) NOT NULL,
  nationality  VARCHAR(100),
  birth_year  INT
);

CREATE TABLE books (       -- book data lives HERE only
  id          SERIAL PRIMARY KEY,
  title       VARCHAR(255) NOT NULL,
  genre       VARCHAR(80),
  year        INT,
  available   BOOLEAN DEFAULT TRUE,
  author_id   INT REFERENCES authors(id) -- ← link, not copy
);

-- Benefits:
-- ✓ Change author nationality once → affects all books
-- ✓ Can store an author before they have any books
-- ✓ Deleting a book never deletes the author
-- ✓ No redundant data

-- The 3 Normal Forms in plain English:
-- 1NF: Every column holds ONE value (no comma-separated lists)
-- 2NF: Every non-key column depends on the WHOLE primary key
-- 3NF: No column depends on another non-key column
```

For this bootcamp, always aim for 3NF. In practice: if you find yourself storing the same information in multiple places, that is a sign you need another table.

SQL Queries — Filter, Join & Aggregate

Using the tables you created in Activity 1, run these queries in pgAdmin Query Tool:

FILTERING:

1. SELECT all available books
2. SELECT books published after 2000, ordered by year descending
3. SELECT books where title contains the word "code" (case-insensitive, use ILIKE)
4. SELECT all books that have NOT been returned yet (loans.returned_at IS NULL)

JOINS:

5. SELECT title, genre, full_name AS author FROM books
INNER JOIN authors ON books.author_id = authors.id
6. SELECT title, full_name, member_name, loaned_at FROM loans
INNER JOIN books ON loans.book_id = books.id
INNER JOIN authors ON books.author_id = authors.id

AGGREGATES:

7. Count how many books exist per genre (GROUP BY genre)
8. Find the author with the most books (COUNT + GROUP BY + ORDER BY DESC + LIMIT 1)

UPDATE & DELETE:

9. UPDATE book id=3: set available = FALSE
10. INSERT a new loan for book id=1, member_name = 'Ana Reyes'
11. SELECT * FROM loans to confirm the new row appears

Bonus: Write a query showing all books and whether they are currently on loan
(hint: LEFT JOIN loans WHERE returned_at IS NULL)

Subqueries & CTEs

Advanced querying — using query results as inputs to other queries

Subqueries

```
-- A subquery is a SELECT inside another query

-- Find books written by British authors (subquery in WHERE)
SELECT title, year
FROM books
WHERE author_id IN (
    SELECT id FROM authors
    WHERE nationality = 'British'
);
-- The inner SELECT runs first, returns a list of ids,
-- then the outer SELECT filters by those ids

-- Scalar subquery – returns a single value
SELECT title,
       year,
       (SELECT AVG(year) FROM books) AS avg_year,
       year - (SELECT AVG(year) FROM books) AS diff_from_avg
FROM books
ORDER BY diff_from_avg DESC;

-- EXISTS – check if a subquery returns any rows
SELECT a.full_name
FROM authors AS a
WHERE EXISTS (
    SELECT 1 FROM books AS b
    WHERE b.author_id = a.id
    AND b.available = TRUE
);
-- Returns authors who have at least one available book
```

CTEs — Common Table Expressions

```
-- CTE = a named temporary result set
-- Makes complex queries readable
-- Defined with WITH, used like a table

-- Simple CTE
WITH available_books AS (
    SELECT * FROM books
    WHERE available = TRUE
)
SELECT
    ab.title,
    a.full_name AS author
FROM available_books AS ab
INNER JOIN authors AS a ON ab.author_id = a.id;

-- Multiple CTEs (chain them with commas)
WITH
    programming_books AS (
        SELECT * FROM books WHERE genre = 'Programming'
    ),
    british_authors AS (
        SELECT * FROM authors WHERE nationality = 'British'
    )
SELECT
    pb.title,
    ba.full_name AS author
FROM programming_books AS pb
INNER JOIN british_authors AS ba
    ON pb.author_id = ba.id;

-- CTE vs Subquery: when to use which?
-- CTE: complex logic, reused multiple times, readable
-- Subquery: simple, one-time use, shorter
```

CTEs are one of the most important SQL features for readability. If your query is over 10 lines, break it into CTEs. Day 7: EF Core can translate CTEs using LINQ FromSqlRaw().

Student Enrollment Database — Design, Build & Query

1

Part 1 — Schema Design & Creation (30 min)

Create the following tables: `students` (`id`, `first_name`, `last_name`, `year` CHECK between 1 and 4, `gender`, `enrolled` DEFAULT true, `created_at`), `programs` (`id`, `program_name` UNIQUE), `sections` (`id`, `code` UNIQUE, `year`, `program_id` FK), `student_sections` (`id`, `student_id` FK, `section_id` FK, `enrolled_at` DEFAULT NOW()), and `student_grades` (`id`, `student_id` FK, `grade` CHECK between 0 and 100). Ensure that programs are separated into their own table, grades are not stored as arrays or lists, and student-to-section relationships are normalized using a junction table. Add all necessary PRIMARY KEY, FOREIGN KEY, UNIQUE, DEFAULT, and CHECK constraints to maintain data integrity and ensure the database is in 3NF.

2

Part 2 — Seed Data & INSERT (20 min)

INSERT 4 students, 3 programs, 4 sections, and assign students to sections using the `student_sections` table. Add at least 2–3 grades per student using the `student_grades` table. Use RETURNING to capture generated ids during insertion. After inserting all records, verify the data using SELECT COUNT(*) queries on each table including `students`, `programs`, `sections`, `student_sections`, and `student_grades`.

3

Part 3 — Queries (40 min)

Write 8 SQL queries for the Student Enrollment System. The queries should include displaying all students with their program names using JOIN, displaying students with their assigned section codes, retrieving all currently enrolled students, counting the number of students per section using GROUP BY, counting the number of students per program, computing the average grade of each student using AVG, finding students who are not assigned to any section using LEFT JOIN with IS NULL, and finding sections with no students assigned.

4

Part 4 — Indexes & Constraints (30 min)

Add indexes on `students.program_id`, `sections.program_id`, `student_sections.student_id`, `student_sections.section_id`, and `student_grades.student_id` to improve query performance. Use EXPLAIN ANALYZE before and after creating indexes to compare query execution plans and performance improvements. Ensure that appropriate CHECK constraints are implemented, particularly for year values and grade ranges, to enforce valid and consistent data within the database.

References & Further Reading

Day 6

[1]

PostgreSQL 16 Documentation

postgresql.org · postgresql.org/docs/16/

[2]

pgAdmin 4 — GUI for PostgreSQL

pgadmin.org · pgadmin.org/download

[3]

SQL Tutorial — W3Schools

w3schools.com · w3schools.com/sql/

[4]

PostgreSQL Data Types

PostgreSQL Docs · postgresql.org/docs/current/datatype.html

[5]

Database Normalization Guide

studytonight.com · studytonight.com/dbms/database-normalization.php

[6]

Use The Index, Luke!

Free SQL indexing book · use-the-index-luke.com

[7]

SQL JOIN Visual Explanation

sql-joins.leopard.in.ua · sql-joins.leopard.in.ua

[8]

EXPLAIN ANALYZE Guide

PostgreSQL Docs · postgresql.org/docs/current/sql-explain.html

[9]

Npgsql — .NET PostgreSQL Driver

npgsql.org (used Day 7) · npgsql.org